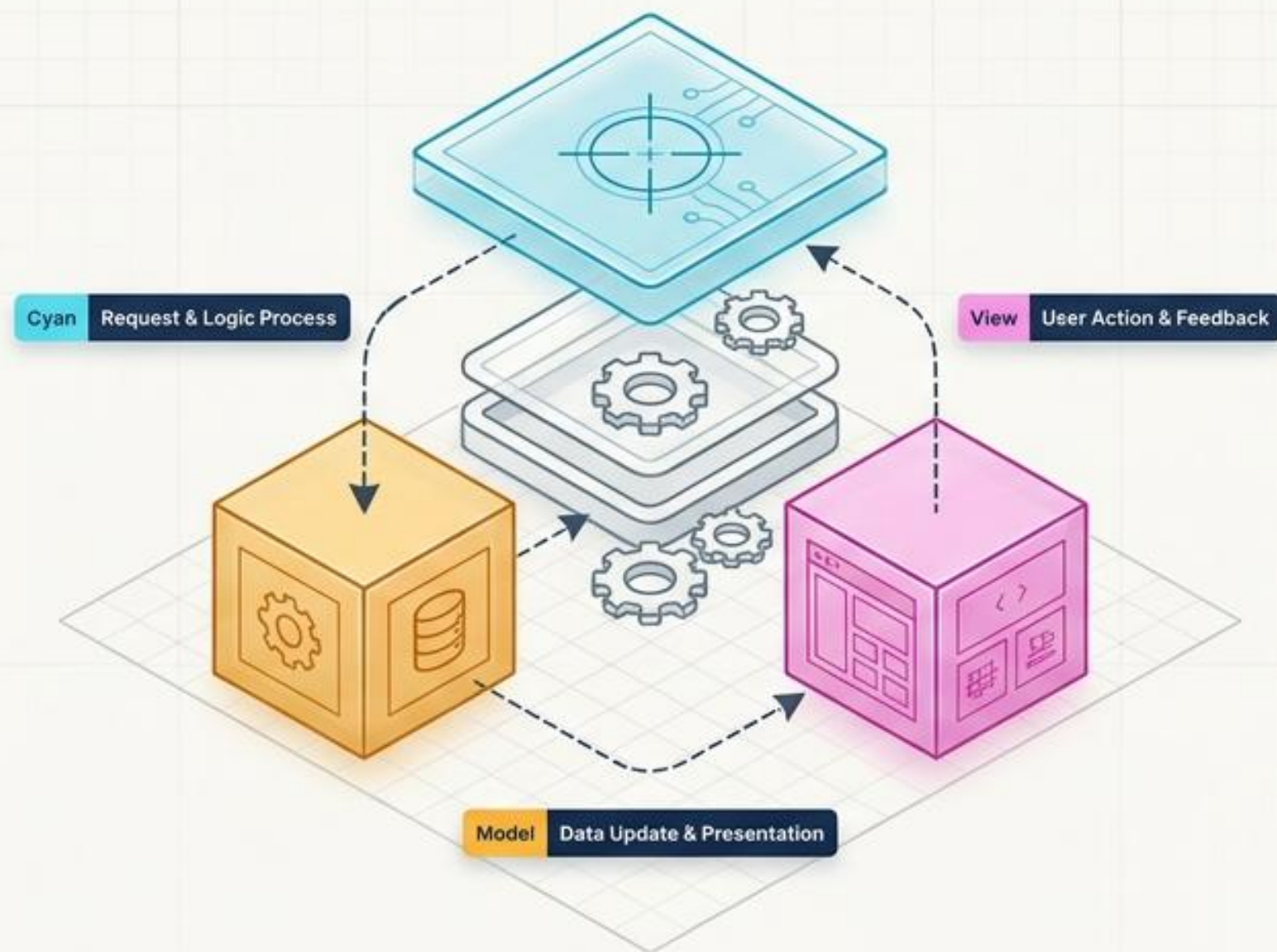




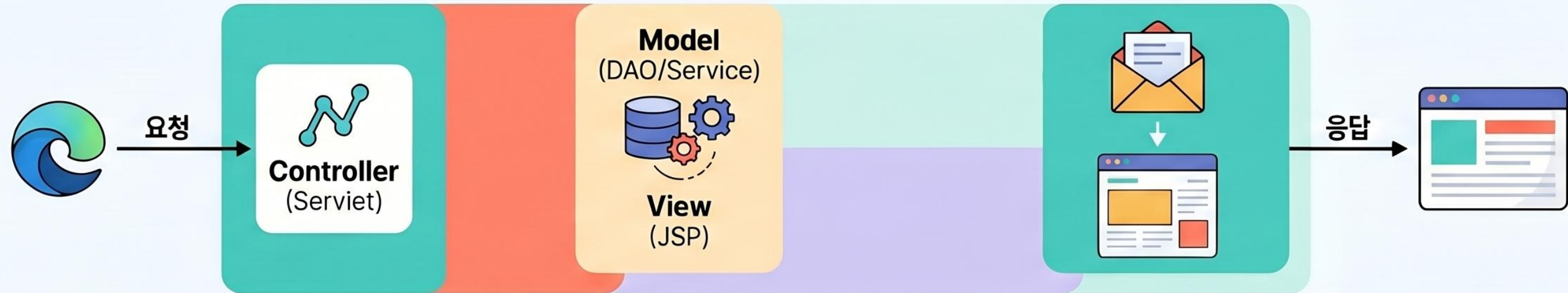
Java 웹 아키텍처 및 데이터 흐름 설계도

MVC, JSP, JSTL, 그리고 XML 파싱 시스템의 구조적 이해

A visual reference guide for modern backend developers.

JSP MVC (Model 2) 구조와 핵심 문법 총정리

MVC 구조와 동작 메커니즘



화면 이동의 정석, Forward vs Redirect

Forward

데이터 유지 시

Redirect (PRG 패턴)

등록/수정 후에는 URL이 바뀜

실무 필수 JSP 문법 및 데이터 관리

EL({})과 JSTL(<c: >)의 조합

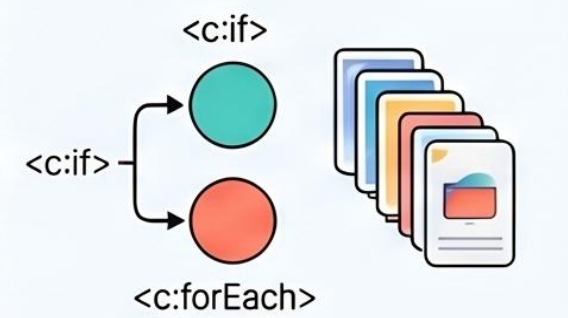
```
${user.name}
...
${user.name}
<c:if test="${not empty list}">
...
</c:if>
```

복잡한 자바 코드 대신 간결한 태그 사용, 가독성 향상

데이터 생명주기를 결정하는 3대 Scope



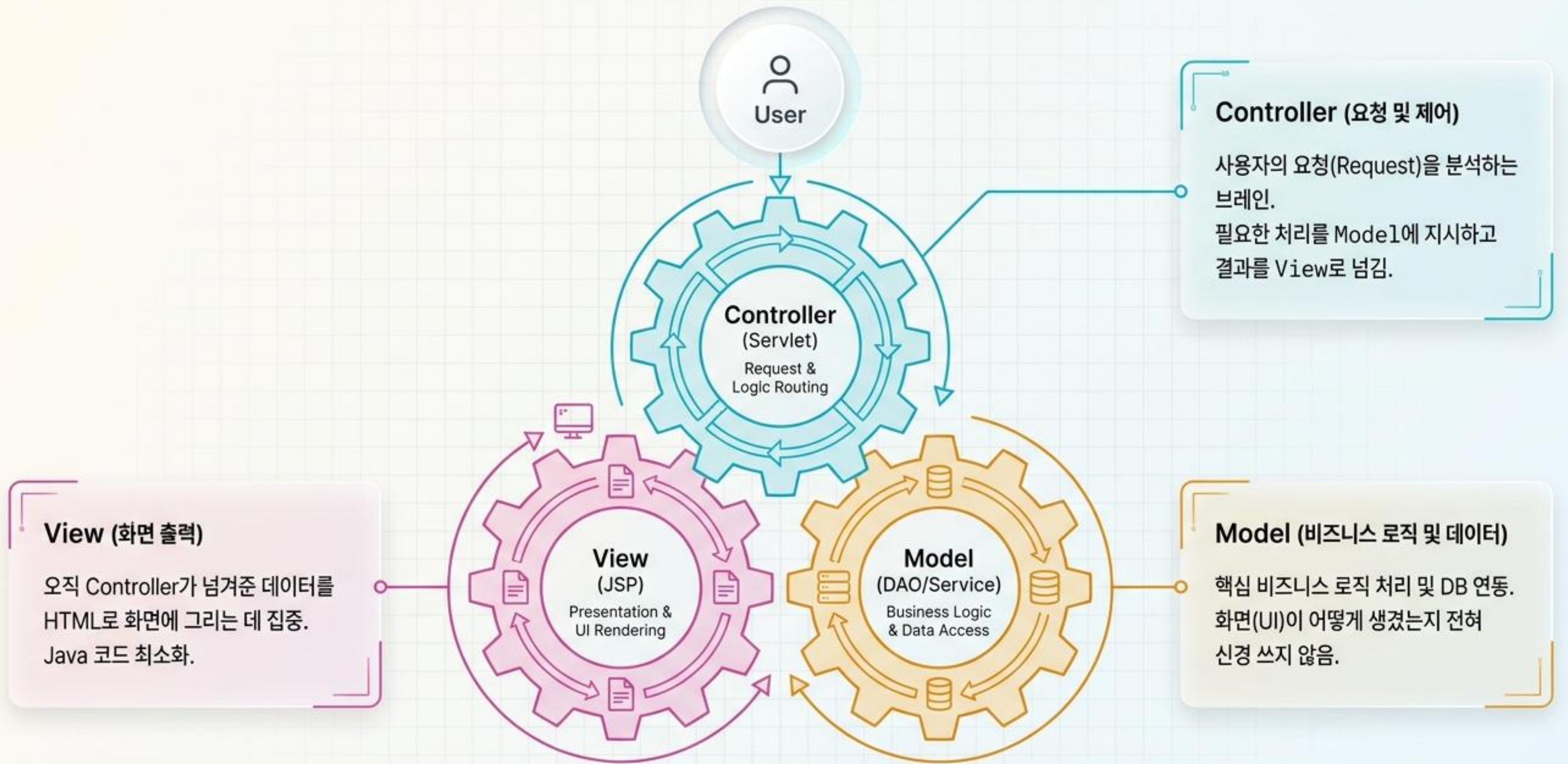
JSTL 핵심 태그: if와 forEach



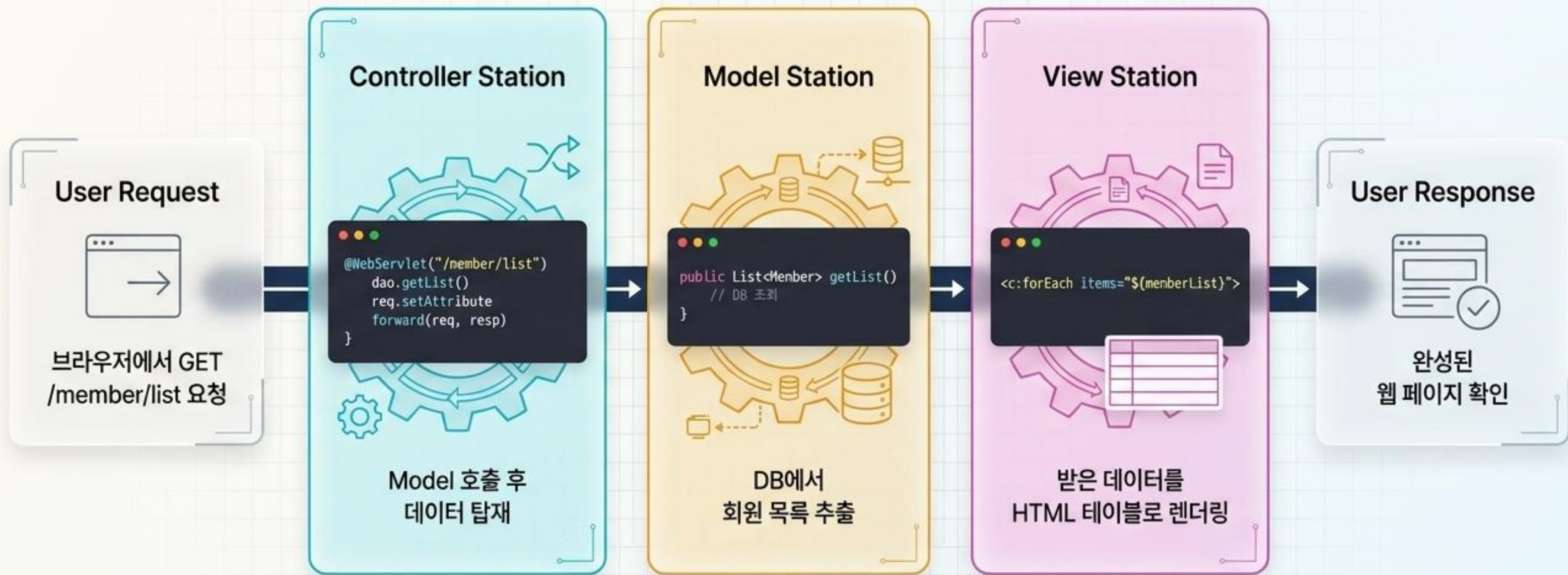
조건문 처리 및 리스트 데이터 반복 출력

XML 데이터 파싱 방식 선택 (DOM vs SAX)

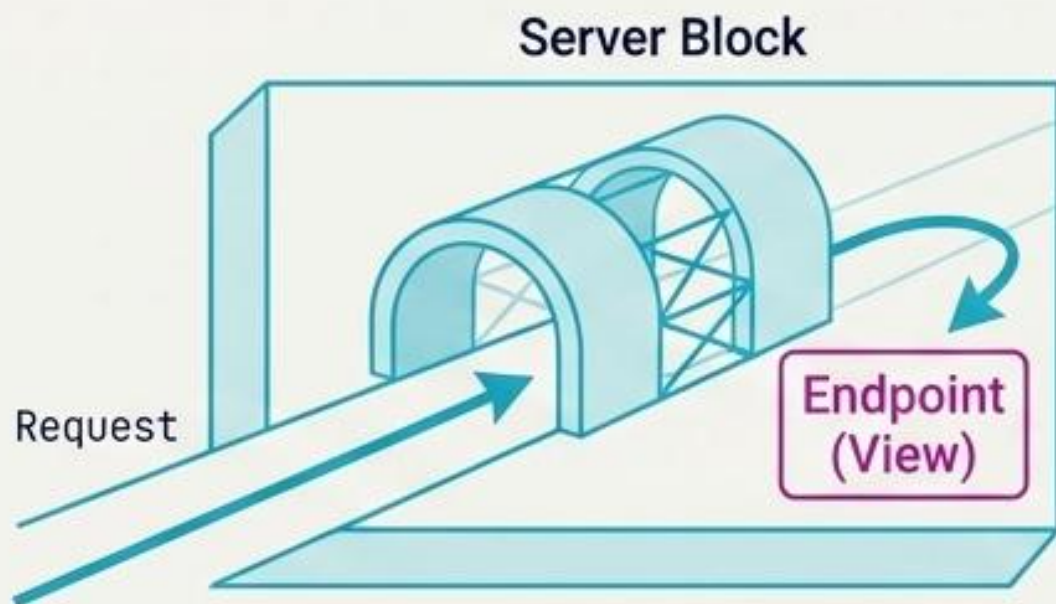
비교 항목	DOM 방식	SAX 방식
작동 원리	전체를 메모리에 로드(Tree 구조)	순차적으로 읽으며 이벤트 발생
메모리 효율	낮음 (대용량 파일에 부적합)	높음 (대용량 파일 처리에 유리)
주요 용도	XML 구조 수정 및 자유로운 탐색	단순 데이터 추출 및 빠른 합계



MVC의 핵심: 화면을 고칠 때는 JSP만, 로직을 고칠 때는 Java 파일만 건드린다. (유지보수의 완벽한 분리)



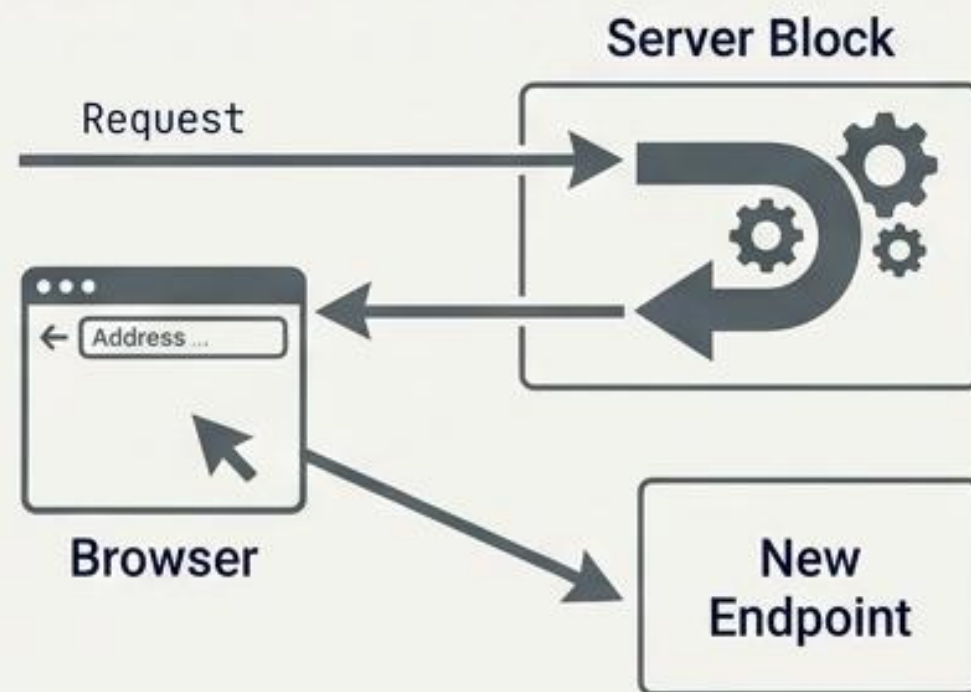
Forward (서버 내부 이동)



```
request.getRequestDispatcher().forward()
```

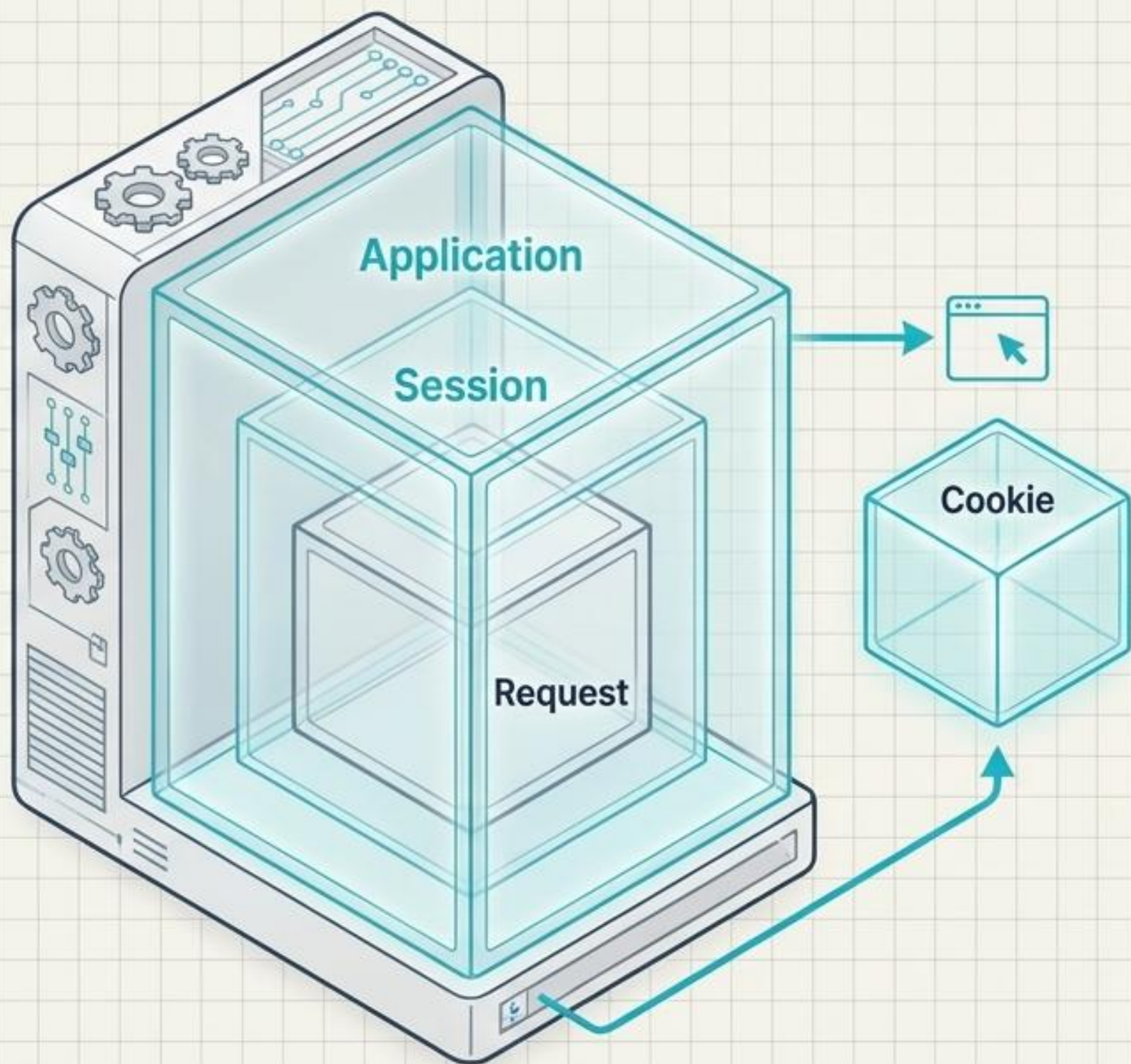
- 🏛️ URL 주소창: 변경 안 됨 (원래 주소 유지)
- 🏛️ Request 객체: 데이터 유지됨 (그대로 전달)
- 🏛️ 실무 사용처: 화면 전환 (조회 결과 보기)

Redirect (브라우저 재요청)



```
response.sendRedirect()
```

- 🏛️ URL 주소창: 완전히 새로운 주소로 변경됨
- 🏛️ Request 객체: 기존 데이터 즉시 소멸 (증발)
- 🏛️ 실무 사용처: 액션 완료 후 튕겨내기 (글쓰기, 로그인 완료)



Data Scope (기술명)	수명 및 용도 (Lifespan & Purpose)
Request (request)	수명: 한 번의 요청-응답 사이 용도: 서블릿에서 JSP로 1회성 데이터 전달 (예: 게시글 목록)
Session (session)	수명: 브라우저 최초 접근 ~ 완전 종료/로그아웃 용도: 사용자 상태 유지 (예: 로그인한 회원 정보, 장바구니)
Application (application)	수명: 서버(Tomcat) ON ~ OFF 용도: 모든 사용자가 공유하는 전역 데이터 (예: 전체 방문자 수 누적)
Cookie (Cookie)	저장 위치: 사용자 PC (브라우저) 용도: 서버 메모리 부담 없이 만료일까지 데이터 유지 (예: 오늘 하루 안 보기, ID 저장)

JSP 지시자(Directive): 컨테이너에게 보내는 작업 지시서

View

```
<%@ page contentType="text/html; charset=UTF-8" import="java.util.List" errorPage="error.jsp" %>
<%@ taglib uri=".../jstl/core" prefix="c" %>
```

```
<html>
  <head>
    ...
```

page 지시자 (환경 설정)

```
<%@ page contentType="text/html; charset=UTF-8"
      import="java.util.List"
      errorPage="error.jsp" %>
```

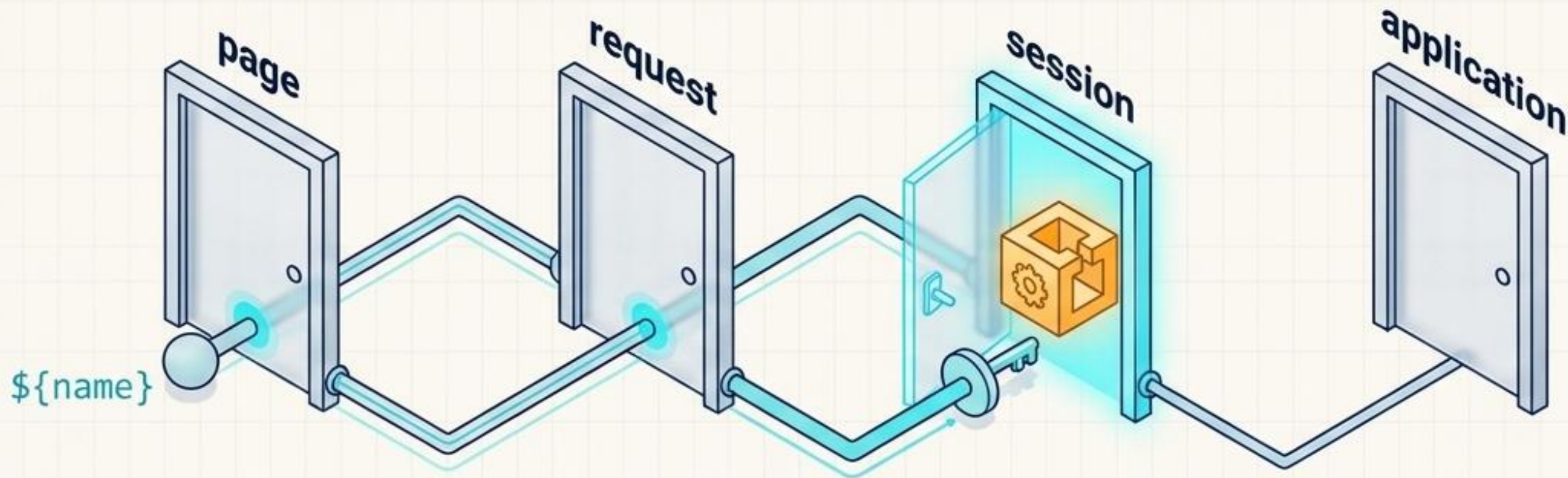
문서 타입(UTF-8), 자바 클래스 임포트,
에러 발생 시 넘길 페이지 지정.

taglib 지시자 (외부 무기 장착)

```
<%@ taglib uri=".../jstl/core" prefix="c" %>
```

JSTL 같은 외부 커스텀 태그를 이 페이지에서
c: 라는 접두사로 쓰겠다는 선언.

EL 묵시적 탐색(Implicit Scope Search): Scope 생략 규칙



EL (Expression Language)의 핵심 규칙

- ⚙️ **EL (Expression Language)의 핵심** : 스크립틀릿(<% %>)을 버리고 `${}` 만으로 데이터를 출력하는 브릿지.
- **자동 Getter 호출** : `${member.name}` 작성 시 내부적으로 `member.getName()`을 자동 실행.
- 📁 **Scope 생략 규칙** : 저장소를 명시하지 않으면 가장 작은 영역(page)부터 가장 큰 영역(application) 순서로 데이터를 자동으로 찾아냄.
- ↳ **파라미터 추출 (param)** : `${param.userId}` = `request.getParameter("userId")` 와 완벽히 동일.

EL 연산자 요약

EL 연산자 요약

비교	<code>== (eq), != (ne), < (lt), > (gt)</code>
산술	<code>/ (div), % (mod)</code>



The Power of empty (★실무 최다 사용)

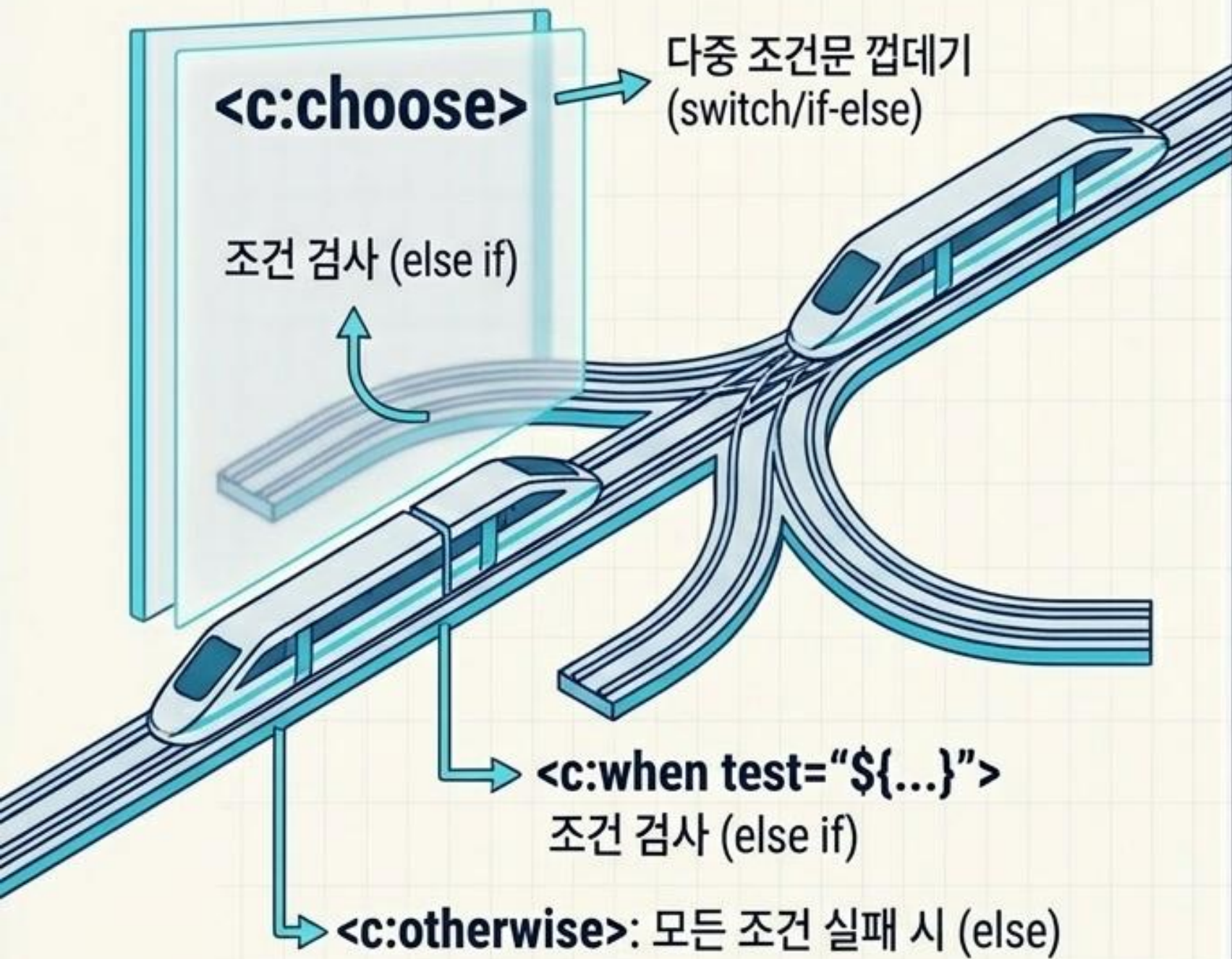
```
${empty sessionScope.loginUser}
```

값이 비어있는지 한 번에 확인하는 마법의 연산자.
다음 3가지 경우 모두 **true**를 반환합니다:

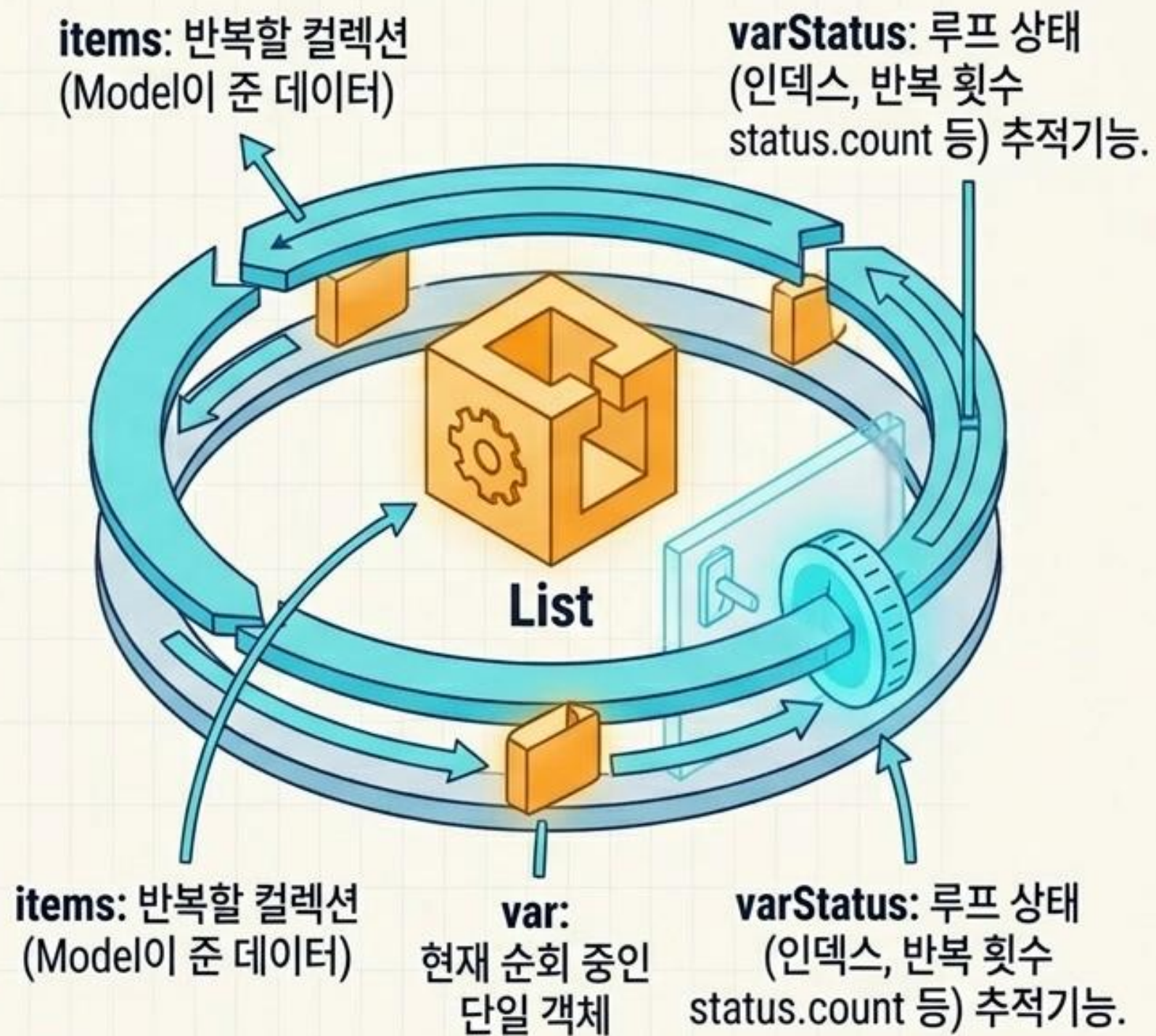
1. 값이 **null**일 때
2. 빈 문자열("")일 때
3. 길이가 0인 배열이나 비어있는 **List**일 때

주로 '로그인이 필요한 서비스입니다' 안내창 출력 시 조건으로 활용.

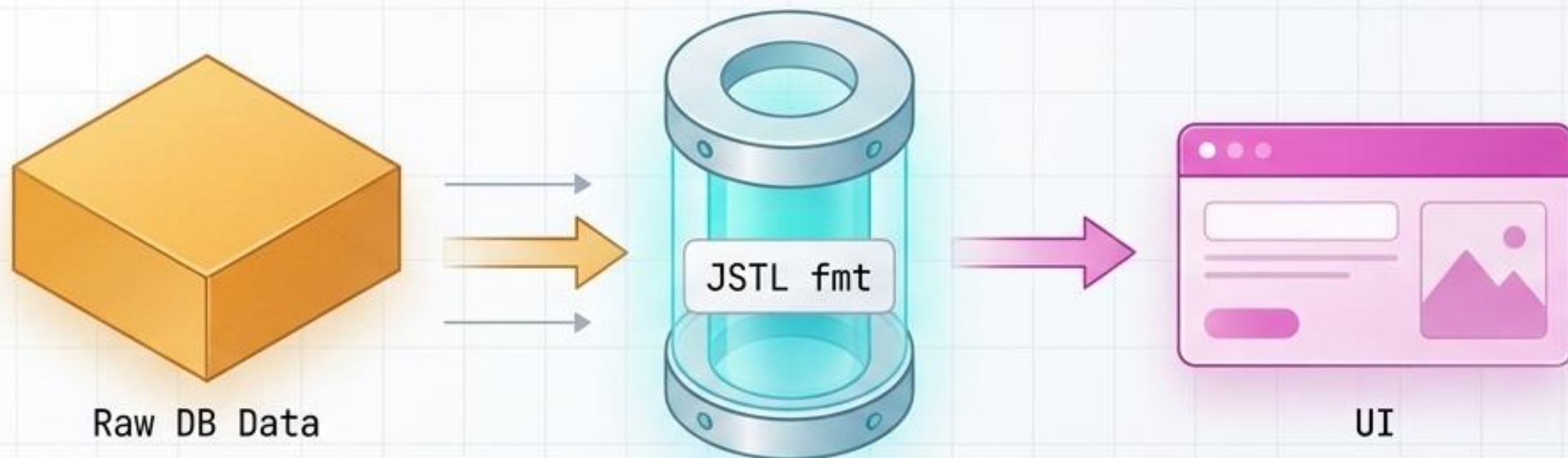
Branching Logic (<c:choose>)



Looping Logic (<c:forEach>)



JSTL fmt 태그 (포매팅)



Date Formatting (<fmt:formatDate>)

Input: java.util.Date

Code: `<fmt:formatDate value="${now}"
pattern="yyyy/MM/dd HH:mm" />`

Output: 2024/10/24 15:30

Number Formatting (<fmt:formatNumber>)

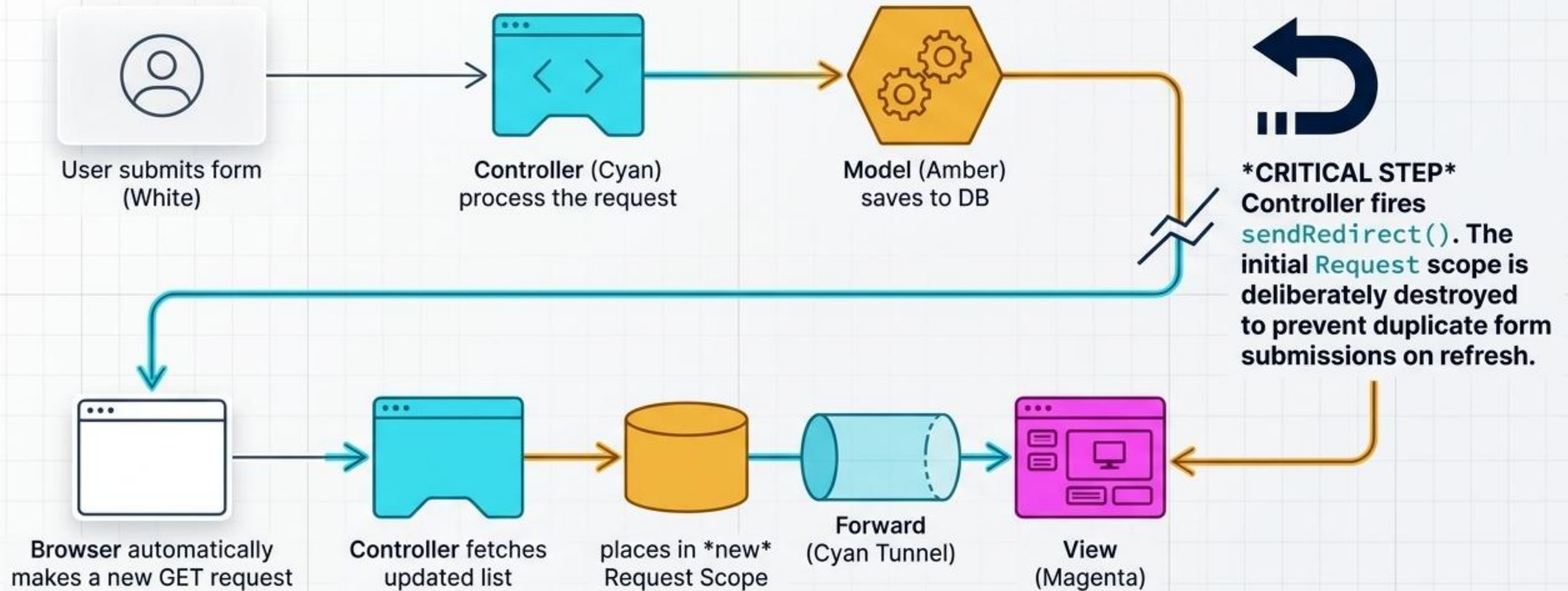
Input: 5000000

Code 1: type="currency" → Output: ₩5,000,000

Code 2: pattern="#,###원" → Output: 5,000,000원

UI에서 복잡한 자바 연산 없이 금액 콤마, 소수점 제어를 선언적으로 완료.

The Master Pattern: PRG (Post-Redirect-Get)



이 패턴을 쓰지 않으면 '새로고침' 시 DB에 똑같은 글이 무한 등록되는 대참사가 발생함.
MVC 구조의 실무 필수 방어 패턴.

외부 데이터 소화하기: XML 파싱

XML 파싱이란?

단순한 텍스트 형태의 XML 문서를 읽어서 Java 시스템이 이해할 수 있는 객체나 데이터 모델로 변환하는 과정.

예시:

```
<member>
  <id>kim</id>
</member>
-> String id = "kim";
```



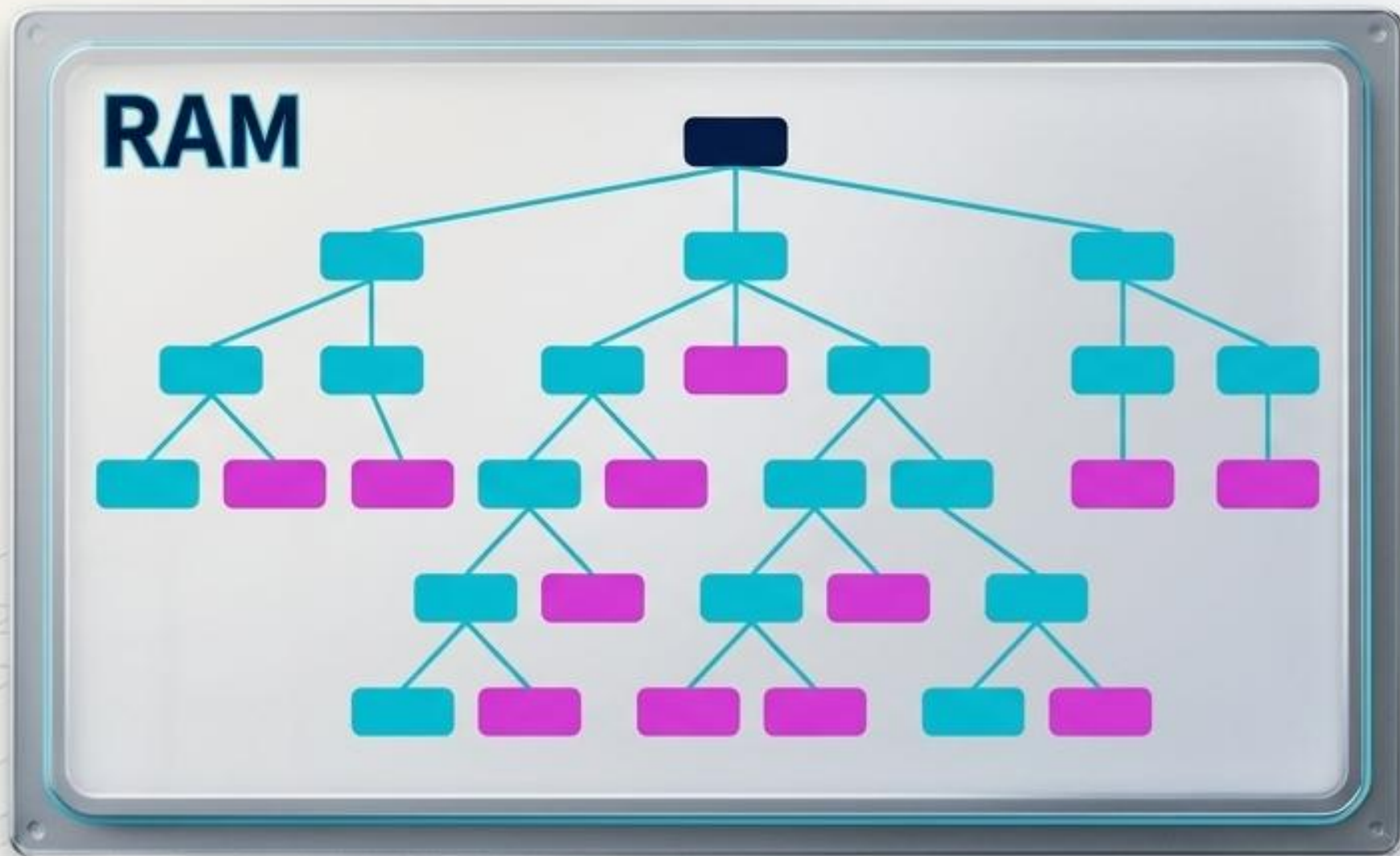
딜레마 (The Dilemma)

데이터의 크기(수십 KB vs 수 GB)와 목적(단순 읽기 vs 구조 수정)에 따라 시스템에 가해지는 부하가 극심하게 달라짐.



DOM 파서 (Document Object Model)

전체 데이터를 메모리에 올리고 Tree를 구축 (DocumentBuilder)



장점 (탐색/수정의 자유)

- 노드의 상하좌우 무한 양방향 탐색 가능.
- 언제든지 노드 데이터 추가/수정/삭제 가능.

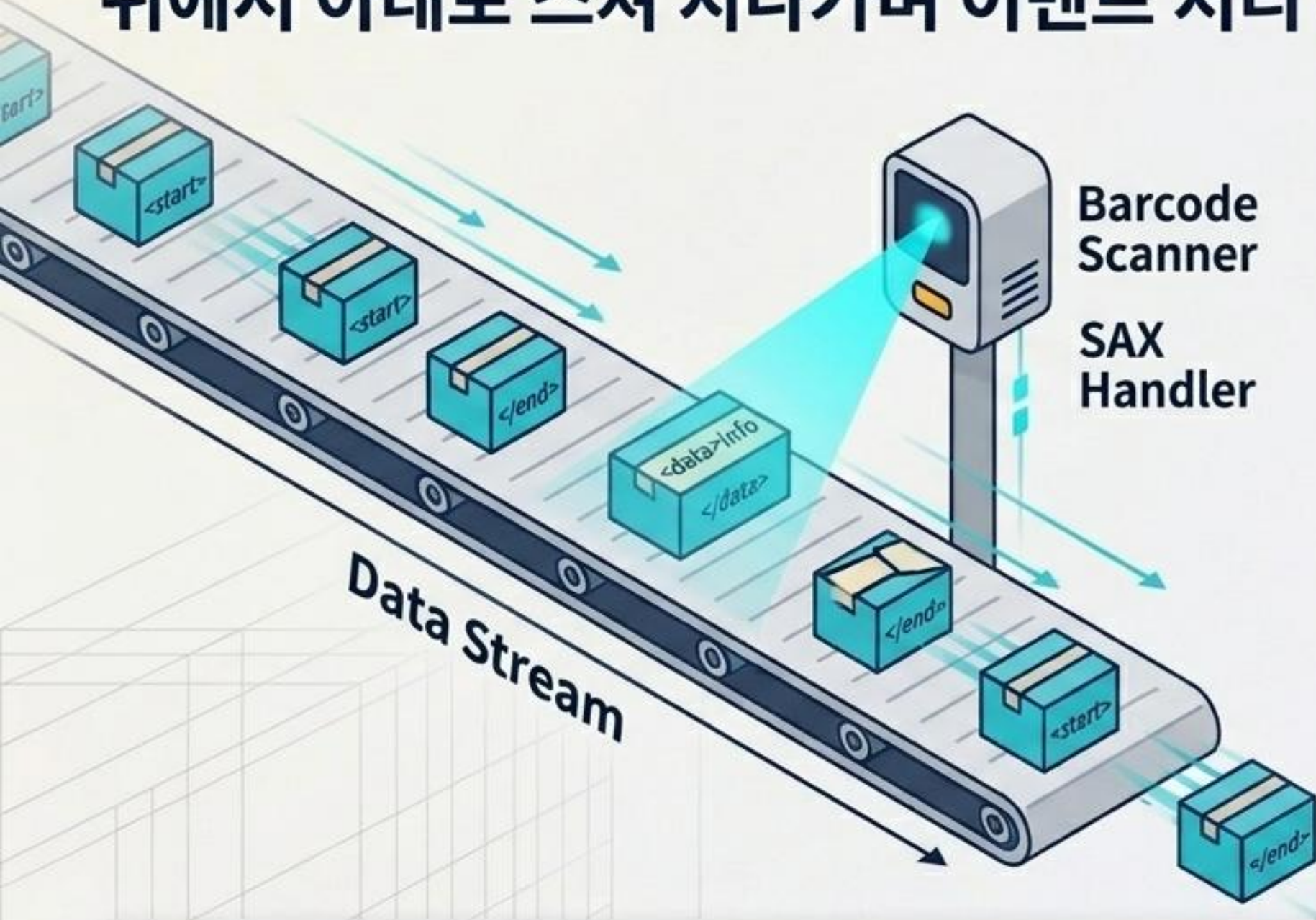
단점 (메모리 폭식)

- 수 GB의 대용량 파일 파싱 시 OutOfMemory 에러 폭발.
- 느린 초기 로딩.

Ideal Use Case: 크기가 작고 내용의 '수정'이 빈번한 설정 파일 등.

SAX 파서 (Simple API for XML)

위에서 아래로 스쳐 지나가며 이벤트 처리 (SAXParser)



장점 (궁극의 메모리 효율)

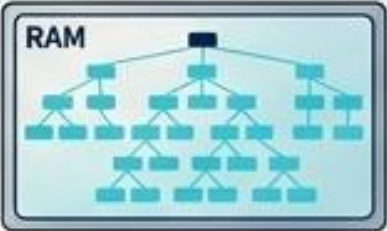
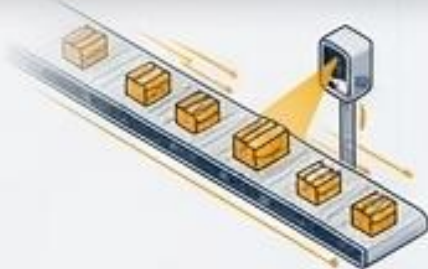
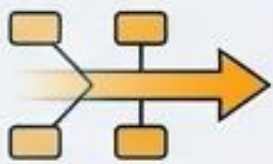
- 한 줄씩 읽고 버리므로 메모리 사용량이 극히 적음.
- 수십 GB 문서도 초고속 처리.

단점 (일방통행)

- 오직 단방향 전진만 가능(뒤로 가기 불가).
- 데이터 수정 절대 불가. 로직 작성이 복잡함.

⚙️ **Ideal Use Case:** 수십 MB~GB 단위의 거대 데이터를 단순 읽어서 DB에 쏟아부을 때.

DOM vs. SAX: The Ultimate Technical Diagnostic Matrix

CATEGORIES	DOM (Document Object Model)	SAX (Simple API for XML)
처리 방식	 Tree 기반 전체 로딩	 Event 기반 스트림 순차 읽기
메모리 효율	 낮음 (메모리 폭식)	 매우 높음 (한 줄 단위)
속도	 상대적으로 느림	 매우 빠름
데이터 수정	 쓰기/수정/삭제 O	 읽기 전용 (Read-only)
탐색 방향	 양방향 (상하좌우 자유)	 단방향 (전진만 가능)

Final Golden Rule: 작은 파일의 구조를 뜯어고칠 때는 DOM을, 대용량 데이터를 초고속으로 흡수할 때는 SAX를 선택하라.